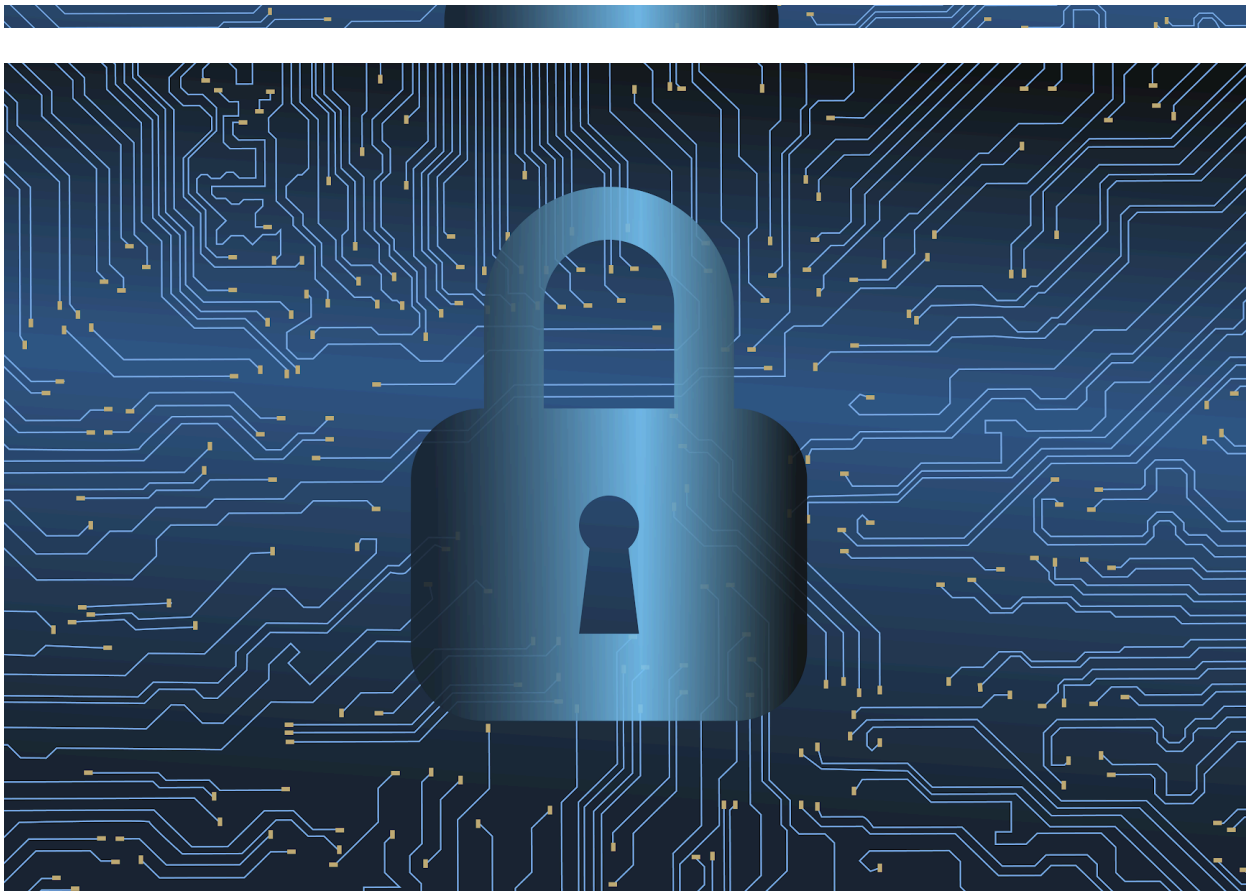


Unit 7 Project: Exploiting Android with Metasploit

10.18.2026

—

Marcus Sanford
2606-FTB-CT-CYB-PT

Overview

This project focuses on setting up a controlled Android security lab using Kali Linux, Android-x86, Metasploit Framework, and Apache2. The goal is to learn how security testing tools are used to examine potential mobile security risks and understand how harmful applications can put a device at risk.

The project uses virtual machines to simulate a testing environment without targeting a real smartphone. Throughout the lab, I documented the setup process, checked network connectivity, and reviewed how files and security tools are used during a controlled assessment.

Goals

- Set up an Android virtual machine to use as the test device.
- Prepare Kali Linux with the tools needed for the lab.
- Learn how Metasploit Framework and Apache2 are used during a security assessment.
- Identify the Kali Linux IP address and understand its role in network communication.
- Understand the risks associated with suspicious applications and connections.
- Document the steps completed, observations, and security lessons learned.

Lab Environment

The lab uses virtual machines to provide a separate environment for learning and security testing.

Tools and Technologies:

- **Kali Linux:** Used to run security testing tools.
- **Android-x86 9.0:** Used as the simulated Android test device.
- **Metasploit Framework:** Used to explore security testing concepts.
- **Apache2:** Used to host files within the lab environment.
- **Oracle VM VirtualBox:** Used to run the virtual machines.
- **Linux VM Images:** Source of the Android virtual machine image.

Scope and Safety

All activities described in this project are intended for a controlled educational lab. The purpose is to understand mobile security risks, learn how testing tools work, and recognize the importance of safe application handling and network monitoring.

Step 1: Download and Deploy the Android Virtual Machine

Objective:

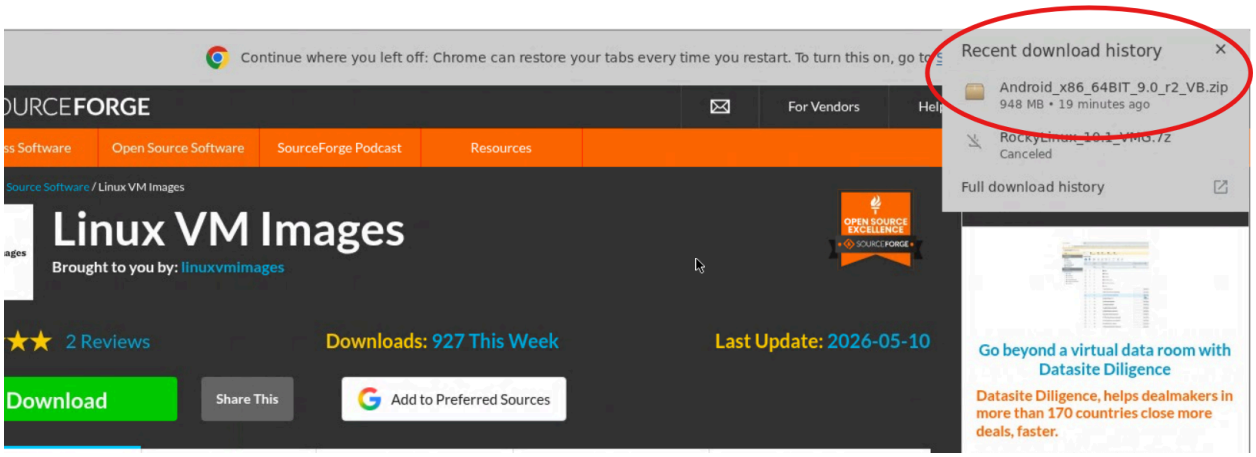
Download and set up an Android-x86 virtual machine to use as the simulated test device for the controlled security assessment.

Tools Used:

- **Android-x86 9.0** — The Android operating system used for the test device.
- **Oracle VM VirtualBox** — Software used to run the virtual machine.
- **Linux VM Images** — Website used to obtain the Android virtual machine image.

1.1 Download the Android VirtualBox Image

Figure 1: Android-x86 VirtualBox Image Download



Action Performed:

I downloaded the 64-bit VirtualBox image provided by the lab instructions. This image was used to create the simulated Android test device for the controlled security assessment.

Key Result:

The Android-x86 virtual machine image was downloaded successfully and was ready to be extracted.

1.2 Extract the Android VirtualBox Image

Figure 2: Android-x86 VirtualBox Image Extracted Using the `unzip` Command

```
labuser@LabVM:~/Downloads$ unzip Android_x86_64BIT_9.0_r2_VB.zip
Archive:  Android_x86_64BIT_9.0_r2_VB.zip
  inflating: Android_x86_64BIT_9.0_r2_VB_LinuxVMImages.com.ova
labuser@LabVM:~/Downloads$
```

Objective:

Extract the downloaded ZIP file to obtain the virtual machine file needed for VirtualBox.

Action Performed:

I used the **`unzip`** command to extract **`Android_x86_64BIT_9.0_r2_VB.zip`**.

The extraction produced the following file:

`Android_x86_64BIT_9.0_r2_VB_LinuxVMImages.com.ova`

Key Result:

The ZIP archive was extracted successfully, producing an OVA file that could be imported into Oracle VM VirtualBox.

1.3 Import and Start the Android-x86 Virtual Machine

Figure 3: Android-x86 Virtual Machine Running



Objective:

Import the Android virtual machine into VirtualBox and confirm that it starts successfully.

Action Performed:

I opened the extracted OVA file and imported it into Oracle VM VirtualBox. After completing the setup, I started the Android-x86 virtual machine.

Observation:

The Android-x86 virtual machine started successfully and displayed the Android environment.

Key Result:

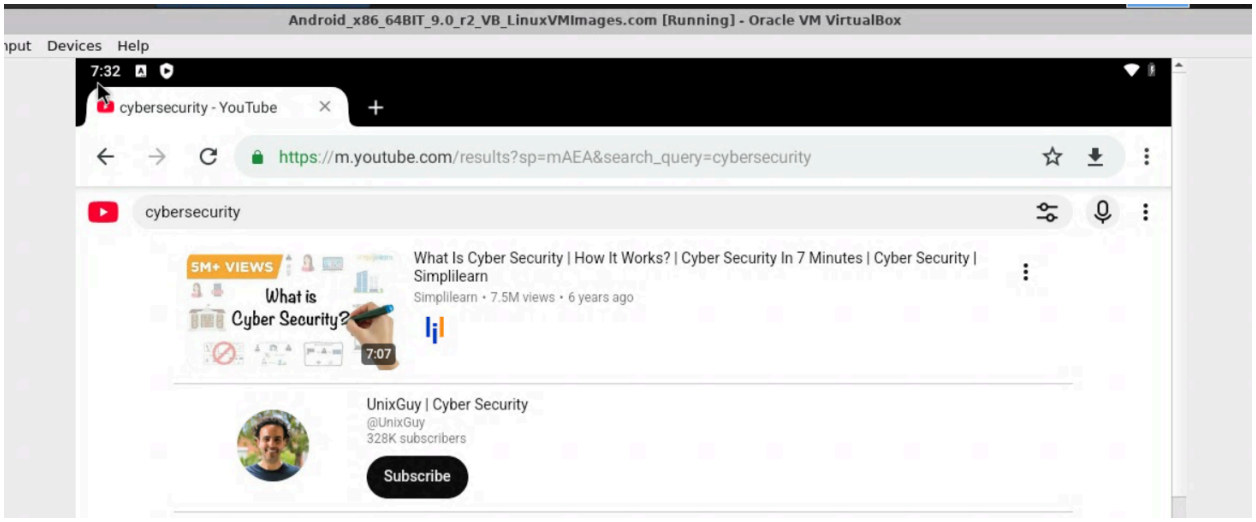
The virtual machine was ready to serve as the simulated Android test device for the remaining lab activities.

Security Significance:

Using a virtual machine provides a separate testing environment where security concepts can be explored without targeting a real smartphone.

1.4 Test Android Internet Connectivity

Figure 4: Android Internet Connectivity Test



Objective:

Check whether the Android virtual machine could access the internet.

Action Performed:

I opened the browser inside the Android virtual machine and navigated to YouTube to test internet connectivity.

Observation:

The YouTube website loaded successfully in the Android browser.

Key Result:

The Android virtual machine accessed YouTube successfully, confirming that internet connectivity was working at the time of the test.

Security Significance:

A device can appear to work normally even when suspicious activity may be happening in the background. This is why it is important to pay attention to the applications installed on a device, the permissions they request, and their network activity.

Step 1 Summary

I downloaded the Android-x86 virtual machine image, extracted the OVA file, imported it into VirtualBox, and confirmed that the Android virtual machine started successfully. I also tested the browser and confirmed that YouTube loaded.

These steps established the Android test environment needed for the next part of the project, where I prepared Kali Linux and the security testing tools.

Step 2: Prepare the Lab Environment

Objectives:

Prepare the Kali Linux virtual machine for a controlled Android security assessment by installing the Metasploit Framework, setting up Apache2, and identifying the Kali Linux network address.

Tools Used:

- Kali Linux
- Metasploit Framework
- Apache2
- Oracle VM VirtualBox

2.1 Metasploit Framework and Apache2 Installation

Objective:

Install and prepare Metasploit Framework and Apache2 on Kali Linux for the controlled security assessment.

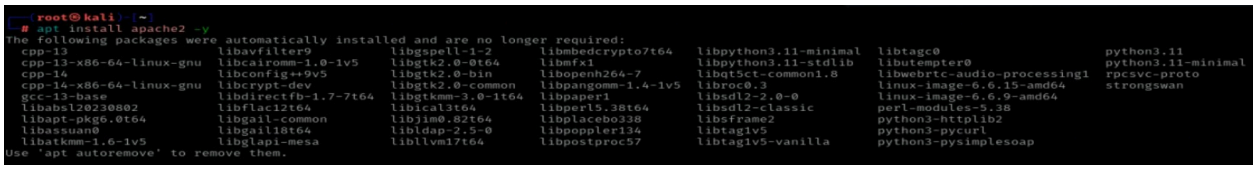
Figure 5a: Metasploit Framework Installation



Action Performed:

I used the **`apt install metasploit-framework -y`** command to install the Metasploit Framework.

Figure 5b: Apache2 Installation



I used the **`apt install apache2 -y`** command to install Apache2.

Key Results:

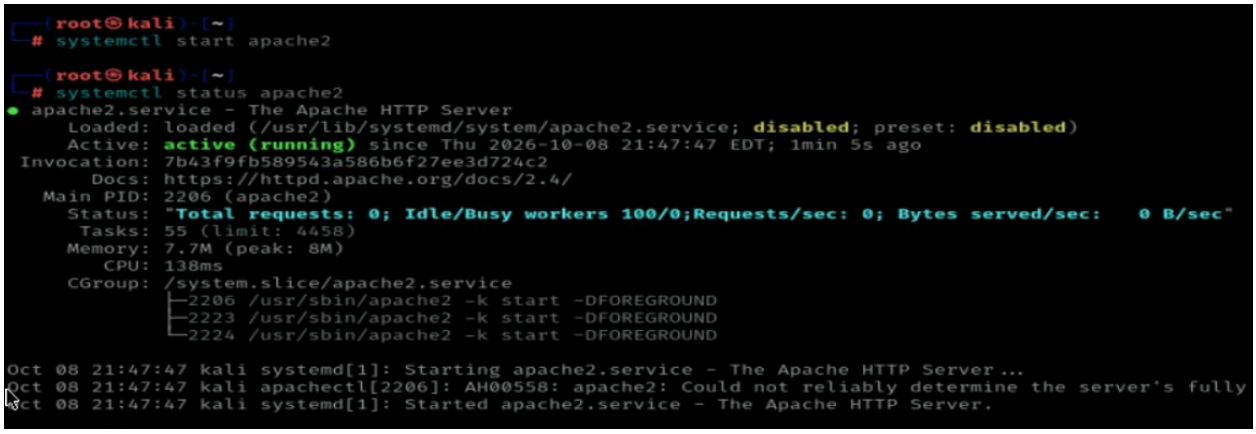
Metasploit Framework and Apache2 were made available for configuration on the Kali Linux virtual machine.

Security Significance:

Metasploit Framework provides tools for security testing, while Apache2 can host web content in the lab. Web servers should be configured carefully to prevent unauthorized access to hosted files.

2.2 Activate and Verify Apache2 web server status

Figure 6: Apache2 Service Status



Objective:

Confirm that the Apache2 web server is running

Action Performed:

I started the Apache2 service and checked its status using the **`systemctl status apache2`** command.

Observation:

The status displayed **`Active: active (running)`**, confirming that Apache2 started successfully.

Key Result:

The service status confirmed that Apache2 was active and running at the time of the check.

2.3 Create the Apache2 Games Directory

Figure 7: Apache2 Games Directory Created



Objective:

Create a directory to hold files used during the controlled Android security assessment.

Actions Performed:

I created a folder named **games** inside **/var/www/html/**, the default Apache2 web directory. I then used the **ls** command to check the directory contents.

Observation:

The **games** directory was created successfully. Its contents were checked during the lab.

Key Result:

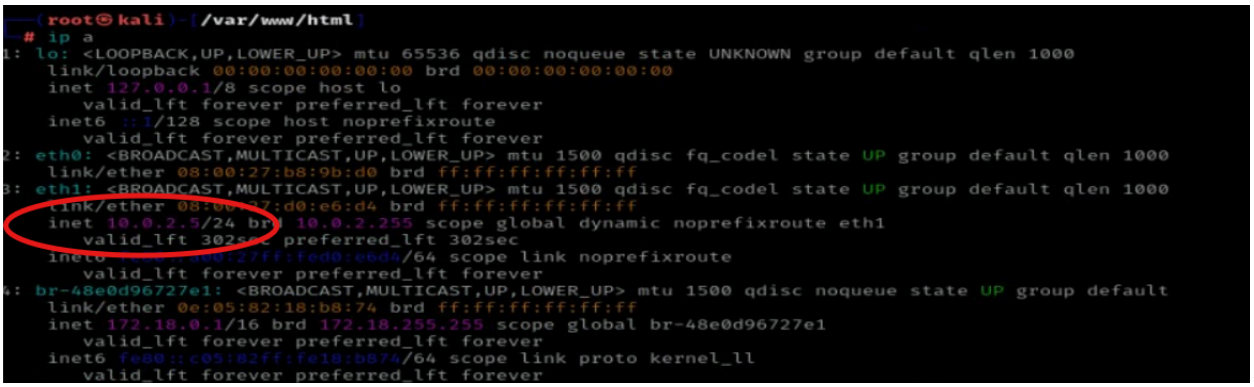
The **games** directory was created under **/var/www/html/** and appeared in the directory listing.

Security Significance:

Web servers can make files available to other systems over a network. This demonstrates how a web server could be used to deliver an application and why downloading files from untrusted sources can create security risks.

2.4 Identify the IP Address

Figure 8: Identify the Kali Linux IP Address



Objective:

Identify the Kali Linux network address for the controlled lab environment.

Actions Performed:

I ran the **`ip a`** command to review the network interfaces and identify the assigned IPv4 address.

Observation:

The Kali Linux interface displayed the IPv4 address **`10.0.2.5.`**

Key Result:

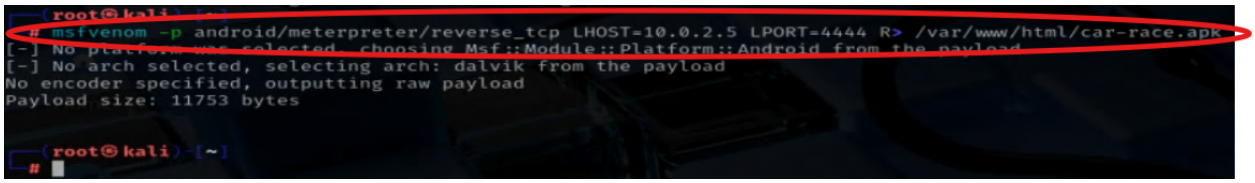
I identified **`10.0.2.5`** as the Kali Linux IPv4 address shown in the lab. Whether another virtual machine can reach this address depends on the virtual network configuration.

Security Significance:

Knowing a system's IP address helps a security analyst understand how devices communicate across a network. Checking network connectivity is an important part of troubleshooting and assessing systems in a controlled lab environment.

2.5 Generate the Android APK File

Figure 9: Android Payload File Generation



Objective:

Generate a file named **`car-race.apk`** using Metasploit's **`msfvenom`** tool as part of the controlled lab.

Action Performed:

I used msfvenom to generate a file named **`car-race.apk`**. The lab configuration referenced the Kali Linux address **`10.0.2.5`** and port **`4444`**.

Security Significance:

Attackers may disguise harmful software as games, useful applications, or software updates to persuade people to install it. This example illustrates the risks associated with applications designed to establish remote access. Users should verify an application's source, review its permissions, and watch for unusual behavior or network activity.

2.6 Verify the Generated File

Figure 10: Verify the Games Directory



Objective:

Check whether the expected output file was present in the lab directory.

Actions Performed:

I navigated to **`/var/www/html/games/`** and used the **`ls`** command to list the directory contents.

Observation:

The terminal output displayed **`car-race.apk`** in the directory listing.

Key Result:

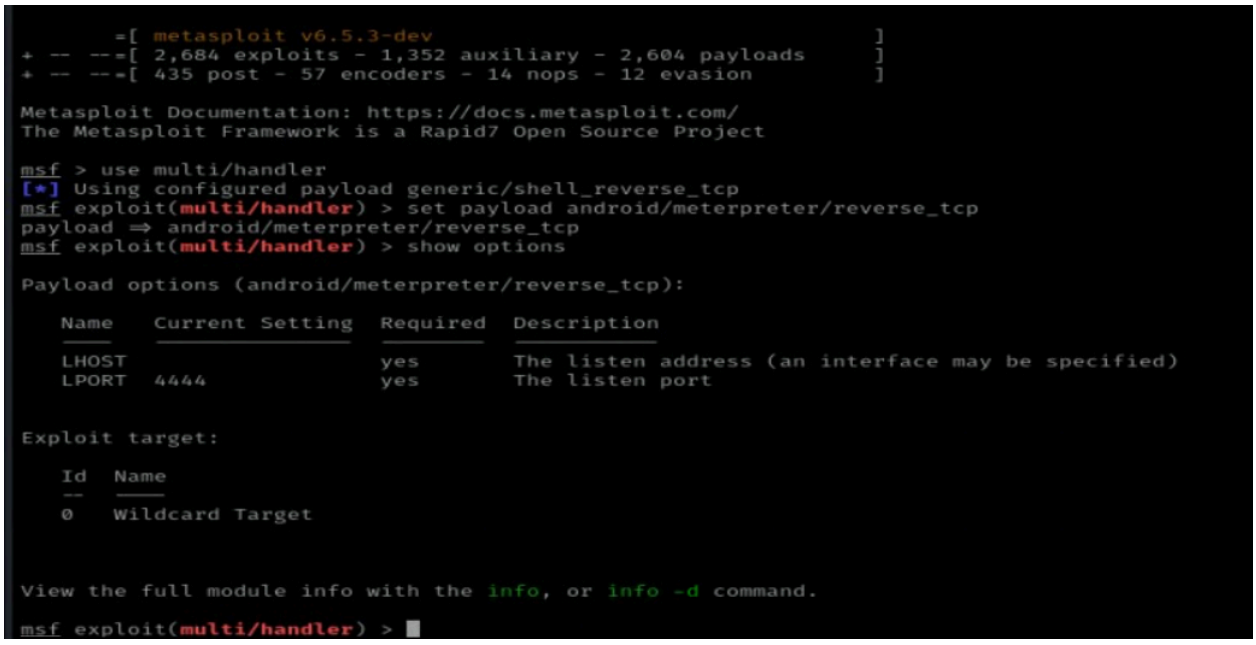
The directory listing showed **`car-race.apk`** in the expected location.

Security Significance:

Files hosted in a web directory may be accessible to other systems, depending on the web server configuration. Verifying files and limiting access to intended content helps reduce security risks.

2.7 Launch and Review the Metasploit Handler Configuration

Figure 11: Metasploit Console and Handler Configuration Options



Objective:

Open the Metasploit Framework console, select the multi-handler module, and review the configuration options associated with the Android reverse TCP payload.

Actions Performed:

I launched the Metasploit Framework console using the **`msfconsole`** command. I then selected the **`multi/handler`** module and specified the Android Meterpreter reverse TCP payload type. Finally, I used the **`show options`** command to display the available configuration settings.

Observation:

I reviewed the Metasploit console workflow and the settings associated with the selected payload. The options included network configuration fields such as **LHOST** and **LPORT**.

Key Result:

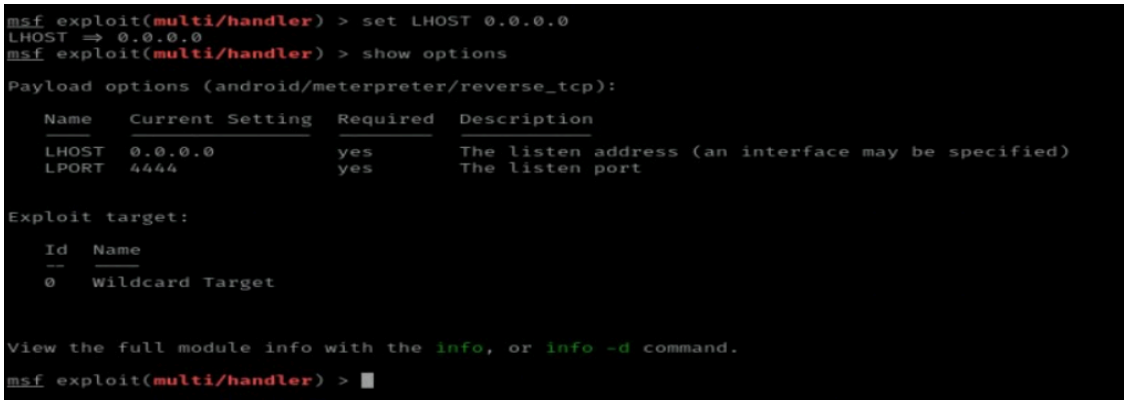
The **LHOST** field was empty, meaning the local host setting had not yet been configured. The **LPORT** field was set to **4444**, which is the port configured for the listener.

Security Significance:

Metasploit uses modules and configuration settings during security testing. A reverse TCP payload initiates a connection back to a configured system. Because this payload can provide remote access, use it only in an authorized, isolated lab environment. From a defensive perspective, monitoring unexpected network connections and investigating suspicious application behavior can help identify potential security threats.

2.8 Review the Metasploit LHOST Configuration:

Figure 12: Configuration of LHOST Metasploit Handler



Action Performed:

I ran the command **`set LHOST 0.0.0.0`** to bind the listener to all available network interfaces.

Observation:

The configuration was updated with **`LHOST`** set to **`0.0.0.0.`**

Key Result:

The configuration displayed **`LHOST 0.0.0.0.`** This is a wildcard IPv4 address that generally represents all local IPv4 interfaces when used as a bind address. It does not identify a specific network interface or guarantee that another device can reach the system.

Security Significance:

Network binding settings affect which local interfaces a service may listen on. Listening on all interfaces can expose a service to more network paths than binding it to a specific interface. Security analysts should verify which interfaces and ports are exposed and restrict access to authorized systems.

Step 2 Summary

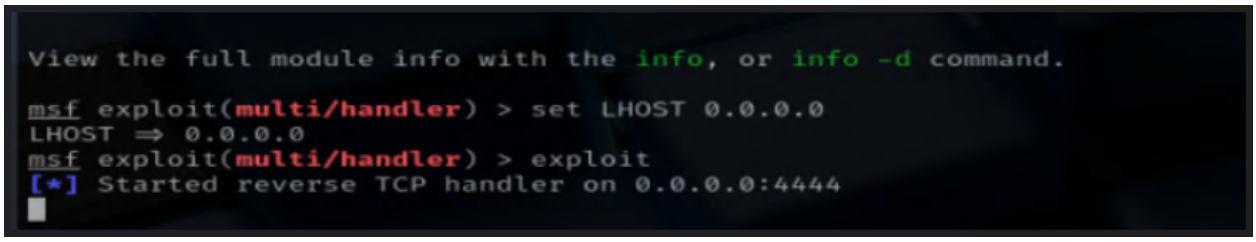
I installed Metasploit and Apache2 in Kali Linux, confirmed that the web server was running, and created a directory to hold the Android application file. I also reviewed the Kali Linux IP address and the Metasploit handler configuration.

These steps prepared the Kali Linux environment and the security tools needed for the Android application assessment in Step 3.

Step 3. Android Application Security Assessment

3.1 Start the Metasploit Handler

Figure 13: Metasploit Reverse TCP Handler Started



Objective:

Start the Metasploit handler and wait for an incoming connection from the Android-x86 virtual machine.

Action Performed:

I ran the **`exploit`** command in the Metasploit console to start the reverse TCP handler and wait for a connection from the target device.

Observation:

The terminal displayed the message **`Started reverse TCP handler on 0.0.0.0:4444`**, indicating that the handler was listening on port 4444.

Key Result:

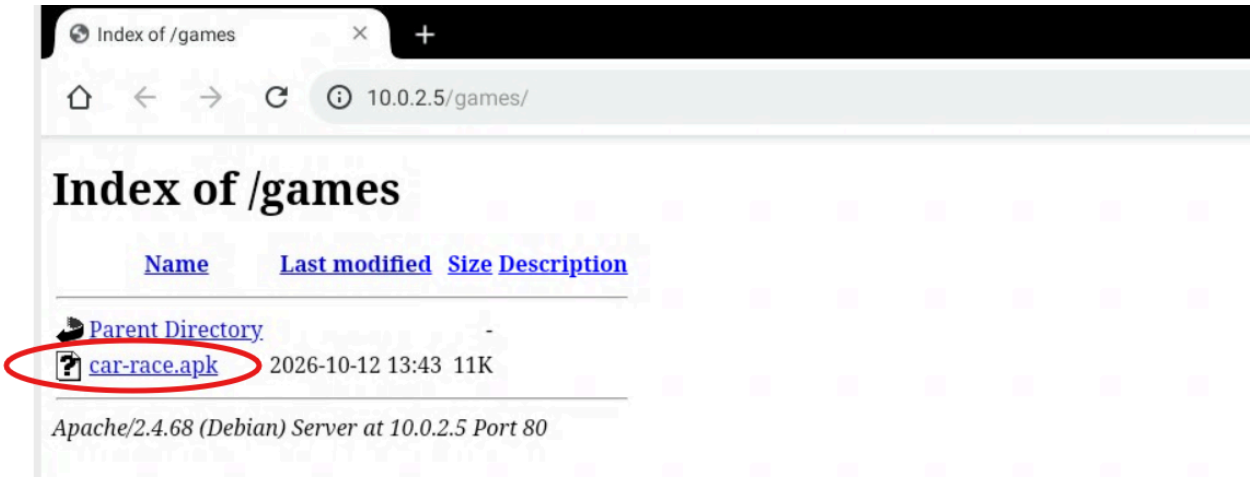
Metasploit confirmed that the handler had started successfully and was waiting for a compatible incoming connection. This message confirmed the listener was running, but it did not confirm that a connection to the Android-x86 virtual machine had been established.

Security Significance:

This exercise demonstrated how a listener waits for incoming connections from a compatible payload. In a malicious situation, this type of setup could be used to establish unauthorized remote access to a device. Understanding how listeners work helps security analysts recognize suspicious network activity, investigate potentially harmful applications, and identify signs of unauthorized access.

3.2 Download and Install the Android Application

Figure 14: Directory containing the harmful APK file.



Objective:

Download the car-race.apk file to the Android device.

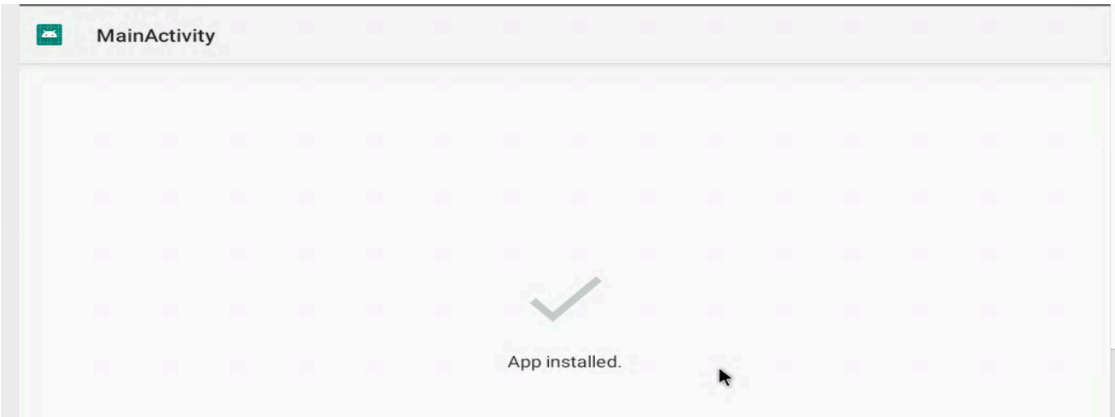
Action Performed:

I entered the web address http://10.0.2.5/games/ in the Android browser to access the directory containing the harmful APK file.

Observation:

The directory page appeared with the car-race.apk file link to download.

Figure 15: MainActivity Application Installed on Android-x86



Key Result:

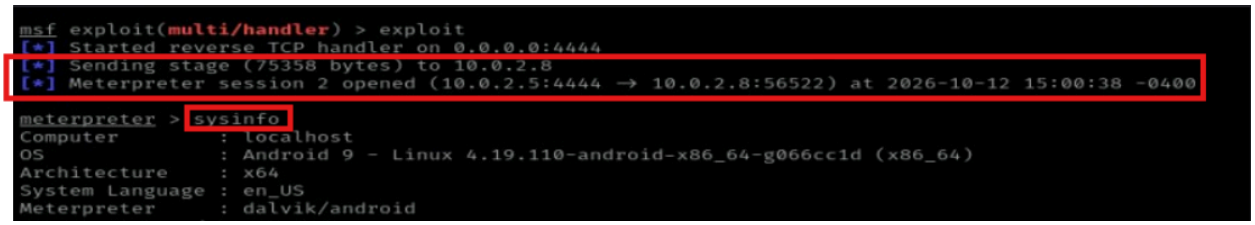
The Android interface showed that an application named **`MainActivity`** had been installed.

Security Significance:

People may overlook security warnings when they trust the sender or are eager to use an application, such as a game, coupon, or work tool. Attackers can exploit trust and urgency to persuade users to download unsafe files. This demonstrates why users should verify an application's source before installing it.

3.3 Confirm Connection and Review Android System Information

Figure 16: Connection to Android-x86 Confirmed



Objective:

Confirm that a connection has been established between Kali Linux and the Android-x86 virtual machine and retrieve basic system information about the target device.

Action Performed:

I switched back to Kali Linux to verify the connection. After confirming that a Meterpreter session had opened, I ran the **`sysinfo`** command to retrieve information about the Android virtual machine.

Observations:

The Metasploit console reported that Meterpreter session 2 had opened, confirming that a connection was established. The connection details showed Kali Linux at **`10.0.2.5:4444`** and the Android virtual machine at **`10.0.2.8:56522`**. The **`sysinfo`** command returned the following information:

- **Computer:** Localhost
- **Operating System:** Android 9
- **Kernel:** Linux 4.19.110-android-x86_64-g066cc1d
- **Architecture:** x86_64
- **System Language:** en_US
- **Meterpreter:** dalvik/android

Key Result:

The successful Meterpreter session and the system information returned by `sysinfo` confirmed that the connection to the Android-x86 virtual machine was established and that the session could retrieve basic device information.

Security Significance

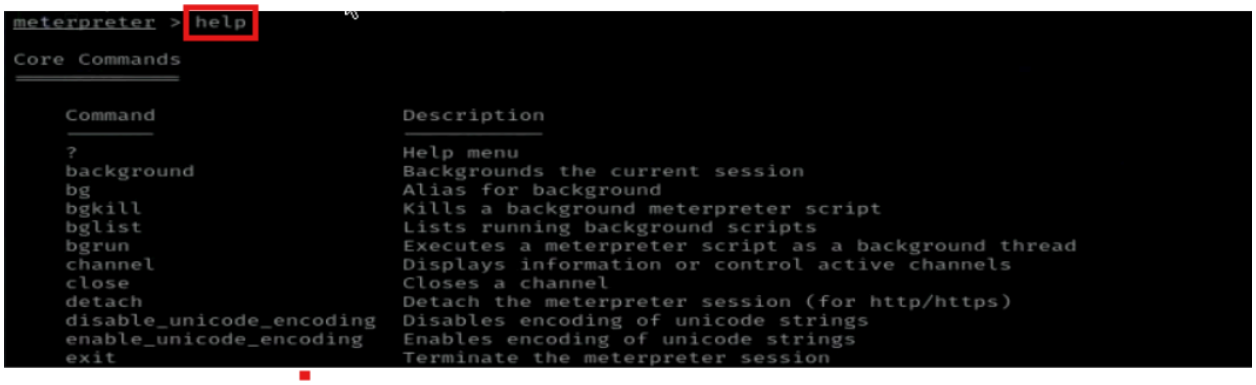
This exercise demonstrated how a successful remote session could allow a connection to retrieve information about a device's operating system and configuration. In a real-world attack, this information could help an attacker understand the target system and plan further actions. From a defensive perspective, monitoring unexpected network connections, investigating suspicious application behavior, and restricting unauthorized access can help identify and prevent potential security incidents.

3.4 Post-Exploitation

Figure 17: Meterpreter Contact Retrieval and Screen-Sharing Commands



Figure 18: Help Command



Objective:

Examine how Meterpreter commands can be used to gather additional information from an Android device after a remote session has been established.

Actions Performed:

Using the established Meterpreter session, I ran the following commands:

- **`dump_contacts`** — Attempted to retrieve contact information stored on the Android-x86 virtual machine.
- **`screenshare`** — Initiated the screen-sharing process to attempt to display the target device's live screen.
- **`help`** - to display a list of available Meterpreter commands and their descriptions.

Observations:

When I executed **`dump_contacts`**, Meterpreter returned the message **`No contacts were found!`**, indicating that no contacts were available for retrieval in the lab environment.

Next, I executed **`screenshare`**. Meterpreter displayed the messages **`Preparing player`**, **`Opening player`**, and **`Streaming`**. The console identified the player location as **`/var/www/html/games/SikIrStC.html`**, indicating that the screen-sharing process had started.

The **`help`** command displayed available Meterpreter commands and descriptions, providing an overview of the additional functions that could be explored through the established session.

Key Result:

- **`dump_contacts`** - retrieval command did not return any contacts.
- **`screenshare`** - command reported that streaming had started
- **`help`** - command provided information about additional Meterpreter capabilities.

Security Significance:

These commands demonstrate how an attacker might gather additional information after gaining access to a device. Saved contacts could potentially be misused for identity theft, phishing, or social engineering against the device owner and their contacts. Screen-sharing capabilities could expose sensitive information displayed on the device. Although no contacts were retrieved in this exercise, the results highlight the potential risks associated with unauthorized access to mobile devices. Defensive measures include reviewing application permissions, installing applications from trusted sources, monitoring suspicious network activity, and preventing unauthorized access to devices.

Step 3 Summary

I accessed the Android application through the Apache2 web server, installed it on the Android virtual machine, and confirmed that a Meterpreter session was established. I also reviewed the Android system information and examined commands related to contact retrieval and screen sharing.

These steps demonstrated how a security assessment can identify the information and device access that may be exposed after a successful connection, highlighting the importance of application permissions and protecting sensitive information.

Security Measures and Lessons Learned

This lab demonstrated how a malicious or untrusted Android application could create security and privacy risks if it were allowed to run on a device. The ability to retrieve device information and attempt screen sharing showed why users and organizations need to protect their devices.

Several security measures can help reduce these risks:

- **Application permissions:** Review the permissions requested by applications and deny access to contacts, the camera, the microphone, or other sensitive information when it is not necessary.
- **Network monitoring:** Monitor network traffic for unusual connections, unexpected communication with unfamiliar systems, or activity on unexpected ports. These signs may help identify suspicious application behavior.
- **Malware detection:** Use reputable mobile security tools and malware-scanning solutions to examine suspicious application files before installation. Avoid installing applications from untrusted sources.
- **Safe application practices:** Install applications only from trusted sources, keep Android and its applications updated, and remove applications that are unnecessary or suspicious.

Overall, this project helped me understand how a security weakness can affect device privacy and why testing in a controlled environment is important. It also showed me how permissions, network monitoring, and malware detection can help defenders reduce the risk of unauthorized access.